

Leaseweb dedicated-server storefront

Project: Angular 19 client demo for a 2–3 day frontend challenge

Stack: standalone components, signals, lazy routes, HttpClient, RxJS, NgRx (quote only), TypeScript, SCSS

Status: demo only — mock OCC JSON, no SAP Commerce Cloud, no `@spartacus/*`, no payment

Not an official Leaseweb product

This note describes the running app: screens, layers, and the files a reviewer can open.

1. Scope

Leaseweb’s public site is a client storefront: pick a dedicated server, configure it, request a quote. First orders are verified, then provisioned — not pay-now checkout.

This repo is that slice only:

In	Out
Four dedicated-server SKUs	Cloud, storage, CDN, 14-SKU catalog
PLP, PDP/configurator, B2B quote	Wizard, compare, multi-item cart
Mock OCC file over HttpClient	Live OCC, CMS, SmartEdit, SSR, OAuth

2. Screens

Path	Screen
<code>/</code>	Home
<code>/servers</code>	Product list (filter by datacenter)
<code>/configure/:sku</code>	Product detail (location, RAM, term, live price)
<code>/quote</code>	B2B quote (company + email)

Routes are lazy-loaded in `src/app/app.routes.ts`. The shell (`header` / `footer` / skip link) stays around `<router-outlet>`.

3. Architecture

```
pages/           home, catalog, configurator, quote
shared/         product card
layout/         header, footer
core/occ/       OCC types, mapper, HttpClient adapter
core/services/  CatalogService, QuoteService (facade)
core/quote-store/  NgRx actions + feature reducer
core/pricing/    monthlyPrice() – pure functions
public/assets/occ/products.json
```

Catalog is HTTP in, UI model out. **Quote** is the only cross-route state, in NgRx. **PDP options and PLP filter** are signals.

That split is intentional: a quote survives PLP → PDP → quote page (cart-like). A location chip does not need the store.

4. Catalog (OCC mock)

`OccProductAdapter` GETs `assets/occ/products.json`. The payload uses OCC-style names (`code`, `price`, `classifications`, `baseOptions`, `pointOfServices`). `occ.mapper.ts` maps that to the UI `Product` / `Location` models.

The HTTP stream uses `catchError`, `startWith(loading)`, and `shareReplay(1)`. A `retry()` re-runs the GET. Home, PLP, and PDP show **loading, error + try again**, or the data.

There is no remote Leaseweb or Commerce API. The adapter is the swap point: the same `load()` could later call `/occ/v2/{baseSite}/products`.

5. Pricing

`monthlyPrice()` in `src/app/core/pricing/price.ts` is a pure function (no `inject`, no Angular):

```
monthly = round((baseMonthly + extraRAM + singaporeBump) × term)
```

- Extra RAM: €16 (or the SKU's `ramDeltaPer32`) per 32 GB above base
- Singapore (`SIN-01`): +€12
- 12-month term: ×0.9

In production this would come from OCC `price`. Here it stays on the client so the PDP can update as the user clicks.

Example: General Purpose EPYC, AMS, 32 GB, month-to-month → **€79**. Same SKU, Singapore, 12 months → **€82**.

6. Quote (NgRx)

Feature store `quote` holds one `QuoteItem` | `null`.

- `QuoteActions.addQuote` / `clearQuote`
- `QuoteService` is a facade: pages call `save()` / `clear()` and read `item`; they do not `dispatch` or `select` the store themselves

The quote page is a reactive form (company + email required). Submit generates a local reference (`LW-2026-...`). Nothing is billed or posted.

Refresh clears the quote (in-memory only).

7. Product detail

`ConfiguratorPage` binds `:sku` with `input()` (`withComponentInputBinding` in `app.config.ts`).
`toObservable(sku)` → `switchMap` into `catalog.product$` → `takeUntilDestroyed()`.

Unknown SKU after a successful catalog load redirects to `/servers`.

8. Tests

Karma/Jasmine, no UI tests:

- `src/app/core/pricing/price.spec.ts` — RAM steps, Singapore, 12-month term
- `src/app/core/occ/occ.mapper.spec.ts` — OCC product → UI model, missing classifications

```
npx ng test --watch=false --browsers=ChromeHeadless
```

9. Limits (accepted)

- Mock OCC file, not Commerce Cloud
 - No `@spartacus/storefront` (no OCC backend to boot against)
 - Frontend price, not OCC `price`
 - One quote line, not a full cart
 - AI used for Angular scaffolding and layout speed; catalog slice, price rules, and store vs signals are specified in this demo
-

10. Files to open first

File	Why
<code>public/assets/occ/products.json</code>	Mock OCC catalog
<code>src/app/core/occ/occ.mapper.ts</code>	OCC → UI
<code>src/app/core/occ/occ-product.adapter.ts</code>	HttpClient + load/error
<code>src/app/core/pricing/price.ts</code>	Monthly estimate
<code>src/app/core/quote-store/quote.reducer.ts</code>	Quote state
<code>src/app/pages/configurator/configurator.page.ts</code>	PDP
<code>src/app/app.routes.ts</code>	Screens